

基于 openGauss 的银行业务系统数据库进阶案例

简介：

通过对数据库原理与技术课程的学习,读者应该已经掌握了按照数据库设计步骤对用户需求进行分析设计,遵循第 3 范式要求完成数据库结构的规范设计,在 openGauss 环境中使用 SQL 语言创建数据库和表,并添加表约束,对合法的业务数据进行增、删、改、查操作,使用事务、视图、索引、存储过程、触发器和游标实现业务处理。本章将利用这些技能完成一个典型的数据库管理系统——银行业务系统的项目开发,要求完成的项目能够满足用户需求中的功能要求。

能力目标：

能够在 openGauss 数据库中实现一个完整的业务系统的数据库全生命周期开发。

1 系统需求分析

任务简介：

本次实验是利用 openGauss 数据库知识,对银行业务系统进行需求分析,绘制 E-R 图,实现数据库的逻辑设计,并展示开发环境,最后对案例所有的知识进行梳理,形成知识图谱,对案例中所需要的数据库技能形成能力图谱,主要如下：

- (1) 需求概述（初级）；
- (2) 开发环境（初级）；
- (3) 数据库逻辑分析（初级）；
- (4) 案例覆盖技能点（初级）。

1.1 需求概述

某银行是一家小型银行企业,有十多万客户。现在要为该银行开发一套银行业务系统,通过计算机对银行日常的业务进行处理,以保证数据的安全性,提高工作效率。

本系统功能结构图如图 1.1 所示。

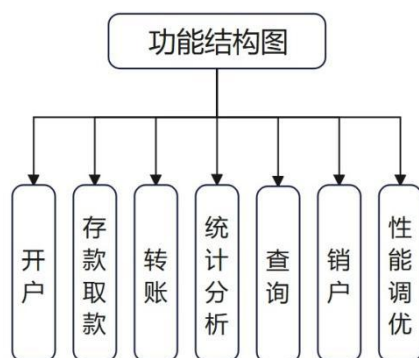


图 1.1 银行业务系统功能结构图

本项目要求根据银行业务需求设计出符合第 3 范式的数据库结构,使用 SQL 语言创建

数据库和表，并添加表约束，进行数据的增、删、改、查，运用触发器、事务、视图和存储过程，按照银行的业务需求实现各项业务（包括日常存款、取款和转账等），并使用 gsql 工具实现性能调优化。

1.2 开发环境

具体开发环境如下。

- （1）数据库：openGauss 5.0.0。
- （2）可视化开发工具：Data Studio。
- （3）终端工具：FinalShell，其安装和建立连接步骤见网址 <https://www.hostbuf.com/t/988.html>。

1.3 数据库逻辑分析

银行为客户提供了各种银行存取款业务，详见表 1-1。

表 1-1 银行存取款业务

业务	描述
活期	无固定存期，可随时存取，存取金额不限的一种比较灵活的存款。
定活两便	事先不约定存期，一次性存入，一次性支取的存款。
整存整取	选择存款期限，整笔存入，到期提取本息，是一种定期储蓄。银行提供的存款期限有 1 年、2 年和 3 年。
零存整取	一种事先原定金额，逐月按约定金额存入，到期支取本息的定期储蓄。银行提供的存款期限有 1 年、2 年和 3 年。
存本取息	存入一笔整数金额后，按约定分期支取利息，到期一次性取出本金。
自助转账	同一币种账户的银行卡之间互相划转。

每个客户凭个人身份证在银行可以开设多个银行卡账户。开设账户时，客户需要提供的开户数据如表 1-2 所示。

表 1-2 开设银行卡账户的客户信息

数据	说明
客户姓名	必须提供。
身份证号	唯一确定客户，由 18 位数字组成或由 17 位数字和 1 位数字或字符组成。
联系电话	11 位的数字。
联系地址	可以选择填写。

银行为每个账户提供一张银行卡，每张银行卡可以存入一种币种的存款。银行卡账户信息如表 1-3 所示。

表 1-3 银行卡账户信息

数据	说明
卡号	银行的卡号由 16 位数字组成。其中:一般前 8 位代表特殊含义，如某总行某支行等。假定该行要求其营业厅的卡号格式为:10103576xxxxxxx。后 8 位一般是随机产生且唯一。
密码	由 6 位数字构成，开户时默认为“888888”。
币种	默认为 RMB。
存款类型	必须选择。
开户日期	客户开设银行卡账户的日期，默认为当日。
开户金额	客户开设银行卡账户时存入的金额，规定不得小于 1 元。
是否挂失	默认为“否”。

客户持银行卡在银行输入密码后，经系统验证身份后办理存款、取款和转账等银行业务。银行规定:每个账户当前的余额不得小于 1 元。

银行在为客户办理业务时，需要记录每一笔账目，账目交易信息如表 1-4 所示。

表 1-4 银行卡交易信息

数据	说明
卡号	银行的卡号由 16 位数字组成。
交易日期	默认为当日
交易金额	必须大于 0 元
交易类型	包括存入和支取两种
备注	对每笔交易作必要的说明

该银行要求这套软件实现银行客户的开户、存款、取款、转账和余额查询等业务，使得银行储蓄业务方便、快捷，同时保证银行业务数据的安全性。

把设计数据库的第一步结果（实体、实体属性，以及实体之间的关系）用 E-R 图表示，如图 1.5 所示。

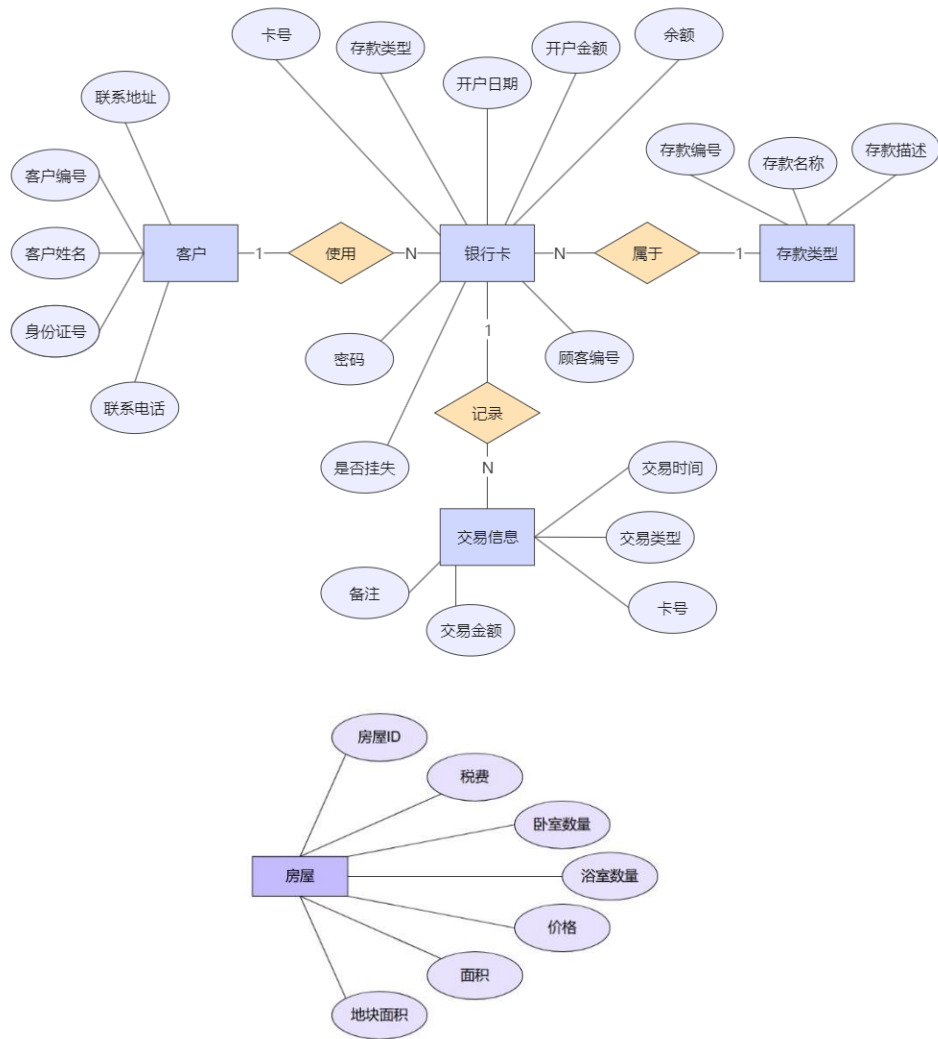


图 1.5 银行业务系统 E-R 图

1.4 案例覆盖技能点

本系统涉及 openGauss 数据库多个知识点和技能，如图 11.2 所示。

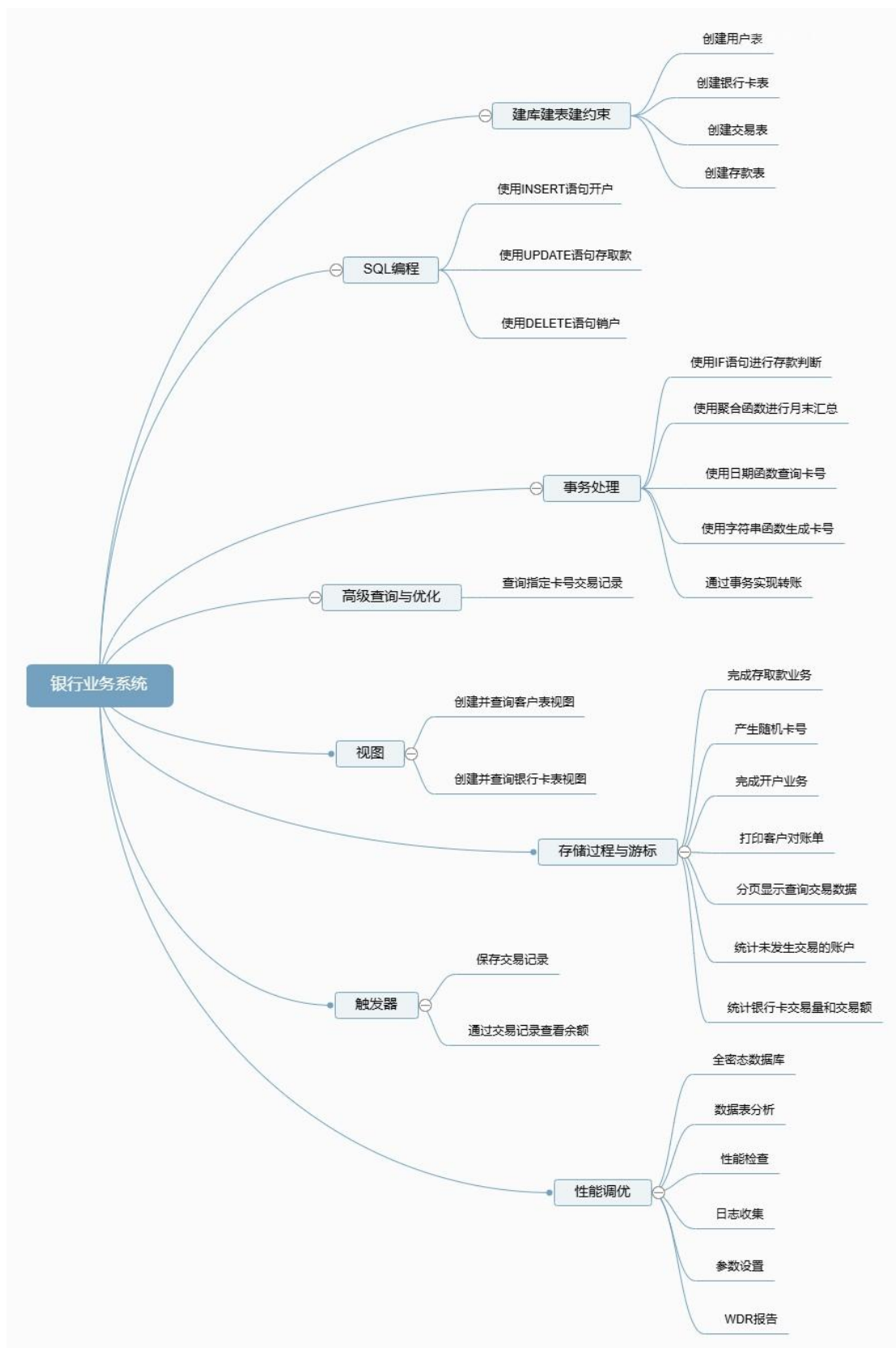


图 11.2 银行业务系统知识点和技能

2 实验进阶 1 银行业务系统数据库表设计

任务简介：

本次实验进阶主要是利用已经完成的数据库需求分析，实现对银行业务数据系统的数据库创建及表设计，主要如下：

- (1) 建立数据库（初级）；
- (2) 建立数据库模式（初级）；
- (3) 建立数据库表（初级）；
- (4) 建立数据库表约束（中级）。

2.1 建立数据库

根据银行业务处理系统的场景描述，本小节分别针对用户（user）、银行卡（card）、交易（trade）和存款（deposit）创建相应的表并建立相应的约束。

启动 FinalShell，单击蓝色文件夹图标，打开连接管理器。

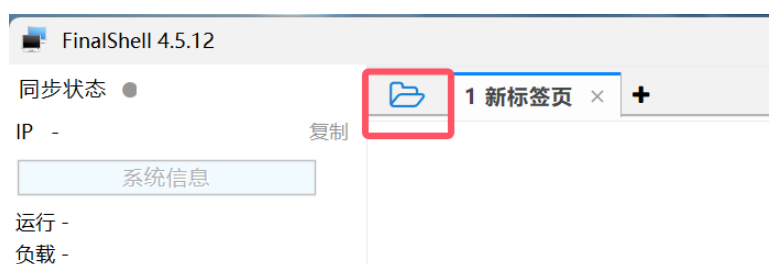


图 2.1 链接管理器

单击白色文件夹图标并选择“SSH 连接”，建立新的连接。

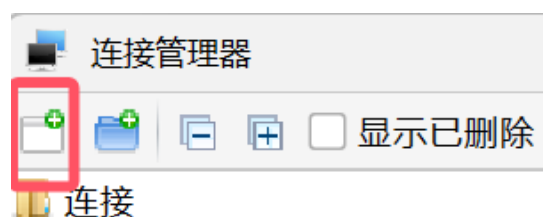


图 2.2 建立 SSH 链接

在新建连接页面，名称可自定义，主机填写华为云 ECS 服务器的公网 IP 地址，端口默认为 22，用户名为 root，密码为服务器 root 用户的密码，填写完毕后点击确定即可与远程服务器建立连接。

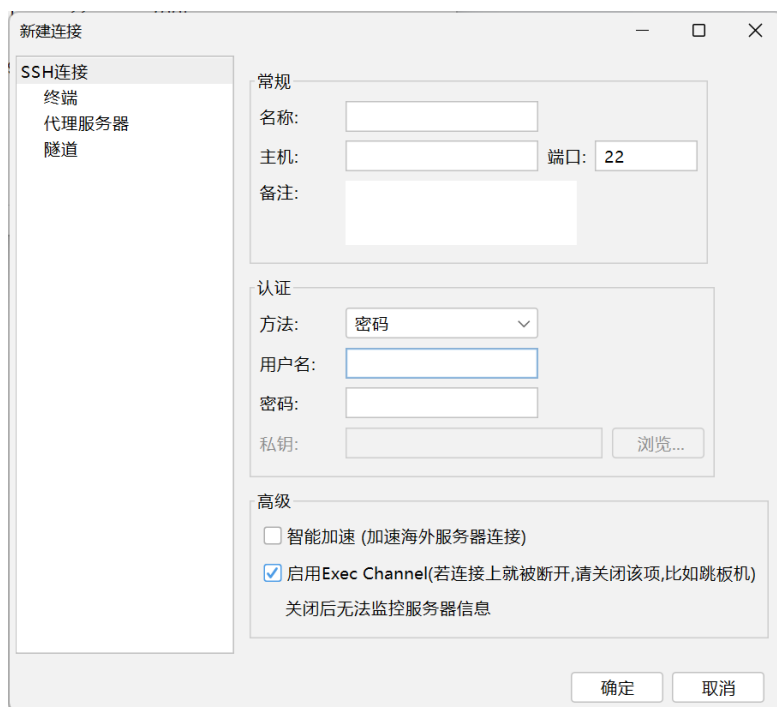


图 2.3 建立远程服务器链接

首先在终端内切换到 openGauss 用户

```
su openGauss
```

```
[root@ecs-marchstorm ~]# su openGauss

Welcome to 4.19.90-2110.8.0.0119.oel.aarch64

System information as of time:  Mon Mar  3 11:15:59 CST 2025

System load:    0.07
Processes:      189
Memory used:    6.8%
Swap used:      0.0%
Usage On:       18%
IP address:     192.168.0.207
Users online:   2
```

图 2.4 切换 openGauss 用户

进入 openGauss 命令行

```
gsql -d postgres -p 5432
```

```
[openGauss@ecs-marchstorm root]$ gsql -d postgres -p 5432
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#
```

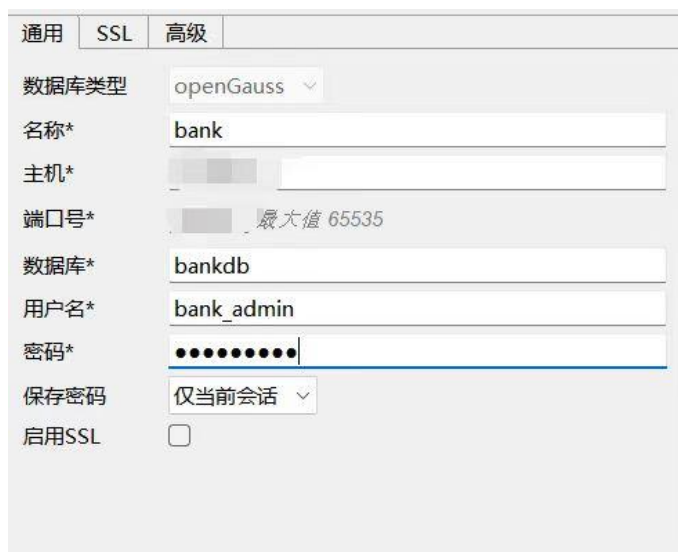
图 2.5 进入 openGauss 命令行

创建银行业务系统数据库并创建数据库用户。

```
CREATE DATABASE bankdb;  
  
CREATE USER bank_admin WITH PASSWORD "Bank@1234";  
  
GRANT ALL PRIVILEGES ON DATABASE bankdb TO bank_admin;
```

2.2 建立数据库模式

切换到 DataStudio 以 bank_admin 的身份登录到数据库 bankdb，如图 11.8 所示。



The image shows the '通用' (General) tab of the Data Studio login window. The fields are filled with the following values: 数据库类型 (Database Type) is 'openGauss', 名称* (Name) is 'bank', 主机* (Host) is empty, 端口号* (Port) is empty with a hint '最大值 65535', 数据库* (Database) is 'bankdb', 用户名* (Username) is 'bank_admin', 密码* (Password) is masked with dots, 保存密码 (Save Password) is set to '仅当前会话' (Current Session Only), and 启用SSL (Enable SSL) is unchecked.

图 2.6 Data Studio 登录界面

在数据库 bankdb 下创建模式 bank_schema，并设置其为搜索路径。首先在 bankdb 终端中输入如下代码，然后选中代码，点击运行按钮即可。

```
CREATE SCHEMA bank_schema;  
  
SET SEARCH_PATH TO bank_schema;
```

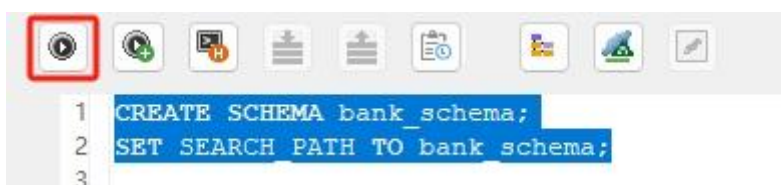


图 2.7 Data Studio 运行代码

2.3 建立数据库表

创建四张表：分别为用户信息表、银行卡信息表、交易信息表和存款类型表。

用户信息表（userInfo）包括如下属性：

- 1) 客户编号（customerID）：主键为客户编号（customerID），且为自增列。
- 2) 客户姓名（customerName）：记录用户的姓名,不能为空。
- 3) 身份证号（PID）：记录用户的身份证号,不能为空。
- 4) 联系电话（telephone）：记录用户的联系电话,不能为空。
- 5) 联系地址（address）：记录用户的联系地址。

```
CREATE TABLE userInfo
(
    customerID SERIAL PRIMARY KEY,
    customerName CHAR(8) NOT NULL,
    PID CHAR(18) NOT NULL,
    telephone CHAR(20) NOT NULL,
    address VARCHAR(50)
);
```

银行卡信息表（cardInfo）包括如下属性。

- 1) 卡号（cardID）：主键
- 2) 币种（curID）：默认为人民币。
- 3) 储蓄编号（savingID）：存款类型的标识符。
- 4) 开卡时间（openDate）：记录银行卡的开卡时间，默认为当前时间。
- 5) 开卡金额（openMoney）：记录开卡时的初始存款金额。
- 6) 账户余额（balance）：记录银行卡的当前余额。
- 7) 取款密码（pass）：记录银行卡的取款密码。
- 8) 是否挂失（IsReportLoss）：记录银行卡是否已挂失，默认为否。
- 9) 客户编号（customerID）：关联用户信息表中的客户编号。

```
CREATE TABLE cardInfo
(
    cardID CHAR(17) PRIMARY KEY,
    curID VARCHAR(10) DEFAULT 'RMB' NOT NULL,
    savingID INT NOT NULL DEFAULT 1,
    openDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    openMoney NUMERIC(18,2) NOT NULL,
    balance NUMERIC(18,2) NOT NULL,
    pass CHAR(6) NOT NULL DEFAULT '888888',
    IsReportLoss VARCHAR(3) NOT NULL DEFAULT '否',
    customerID INT NOT NULL
```

```
);
```

交易信息表 (tradeInfo)包括如下属性。

- 1) 交易时间 (tradeDate): 记录交易发生的时间, 默认为当前时间。
- 2) 交易类型 (tradeType): 记录交易的类型。
- 3) 卡号 (cardID): 关联银行卡信息表中的卡号。
- 4) 交易金额 (tradeMoney): 记录本次交易的金额。
- 5) 备注 (remark): 记录交易的备注信息。

```
CREATE TABLE tradeInfo
(
    tradeDate  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tradeType  CHAR(6) NOT NULL,
    cardID     CHAR(19) NOT NULL,
    tradeMoney NUMERIC(18,2) NOT NULL,
    remark     TEXT
);
```

存款类型表 (deposit)包括如下属性。

- 1) 存款编号 (savingID): 每种存款类型的唯一标识符,且为自增列。
- 2) 存款名称 (savingName): 记录存款类型的名称,不能为空。
- 3) 存款描述 (descrip): 记录存款类型的描述。

```
CREATE TABLE deposit
(
    savingID  SERIAL PRIMARY KEY,
    savingName VARCHAR(20) NOT NULL,
    descrip  VARCHAR(50)
);
```

2.4 建立数据表约束

用户信息表 (userInfo) 的约束如下。

- 1) 身份证号 (PID): 记录用户的身份证号, 唯一, 只能是 18 位。
- 2) 联系电话 (telephone): 记录用户的联系电话, 格式为 11 位数字。
- 3) 联系地址 (address): 记录用户的联系地址, 可选输入。

```
ALTER TABLE userInfo
    ADD CONSTRAINT chk_PID CHECK (CHAR_LENGTH(CAST(PID AS TEXT)) = 18),
```

```
ADD CONSTRAINT chk_telephone CHECK (CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone
AS TEXT), '[^0-9]', '', 'g')) = 11);
```

银行卡信息表 (cardInfo)的约束如下。

1) 卡号 (cardID): 银行的卡号规则和电话号码一样, 一般前 8 位代表特殊含义, 如某总行某支行等。假定该行要求其营业厅的卡号格式为: 10103576xxxxxxxx。

2) 储蓄编号 (savingID): 存款类型的标识符。

3) 开卡金额 (openMoney): 不低于 1 元。

4) 账户余额 (balance): 不低于 1 元, 否则将销户。

5) 取款密码 (pass): 6 位数字, 不能有其他字符。

6) 客户编号 (customerID): 关联用户信息表中的客户编号, 一位顾客可以办理多张卡。

```
ALTER TABLE cardInfo
ADD CONSTRAINT ck_cardid CHECK (
    SUBSTRING(cardID, 1, 8) = '10103576'
    AND SUBSTRING(cardID, 9) ~ '[0-9]{8}$'
),
ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
ADD CONSTRAINT CK_balance CHECK (balance >= 1),
ADD CONSTRAINT CK_pass CHECK (LENGTH(pass) = 6 AND pass ~ '[0-9]+'),
ADD CONSTRAINT FK_customerID FOREIGN KEY(customerID) REFERENCES userInfo(customerID),
ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID) REFERENCES deposit(savingID);
```

交易信息表 (tradeInfo)的约束如下。

1) 交易类型 (tradeType): 记录交易的类型, 只能为“存入”或“支取”。

2) 交易金额 (tradeMoney): 必须大于 0。

```
ALTER TABLE tradeInfo
ADD CONSTRAINT CK_tradeType CHECK(tradeType IN ('存入', '支取')),
ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney > 0);
```

2.5 问题分析

(1) 终端工具 FianlShell 的熟悉与使用, 主要是建立与云服务器的连接时, 主机 IP 地址必须填写公网 IP。

(2) 使用 DataStudio 连接数据库时, 主机端口必须填写自己安装的 openGauss 端口。

3 实验进阶 2 数据录入与修改

任务简介：

本次实验进阶主要是利用进阶 1 已经完成的数据库表设计，实现对银行业务数据的录入与修改，以及简单业务实现，主要如下：

- (1) 录入数据（初级）；
- (2) 修改密码（初级）；
- (3) 挂失银行卡（中级）；

3.1 录入数据

向存款类型表 (deposit)插入存款类型数据。插入数据的示例如下。

```
INSERT INTO deposit (savingName,descrip) VALUES ('活期','按存款日结算利息');
INSERT INTO deposit (savingName,descrip) VALUES ('定期一年','存款期是 1 年');
INSERT INTO deposit (savingName,descrip) VALUES ('定期二年','存款期是 2 年');
INSERT INTO deposit (savingName,descrip) VALUES ('定期三年','存款期是 3 年');
INSERT INTO deposit (savingName) VALUES ('定活两便');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取一年','存款期是 1 年');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取二年','存款期是 2 年');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取三年','存款期是 3 年');
INSERT INTO deposit (savingName,descrip) VALUES ('存本取息五年','按月支取利息');
```

向用户信息表 (userInfo) 和银行卡信息表 (cardInfo)插入用户信息和银行卡信息。插入信息的示例如下。

```
INSERT INTO userInfo(customerid,customerName,PID,telephone,address)
VALUES(1,'John','11000000000000000001','13900000001','江苏省南京市');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(2,'Jane','11000000000000000002','13900000002');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(3,'Michael','11000000000000000003','13900000003');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(4,'Emily','11000000000000000004','13900000004');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(5,'David','11000000000000000005','13900000005');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(6,'Sarah','11000000000000000006','13900000006');
```

```

INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(7,'Robert','110000000000000007','13900000007');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(8,'Jessica','110000000000000008','13900000008');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(9,'William','110000000000000009','13900000009');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(10,'Olivia','110000000000000010','13900000010');

```

```

INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000001','RMB',1,1000.00,1000.00,'888888','否',1);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000002','RMB',2,1.00,1.00,'888888','否',2);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000003','RMB',2,1.00,1.00,'888888','否',3);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000004','RMB',2,1.00,1.00,'888888','否',4);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000005','RMB',3,500.00,500.00,'888888','否',5);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000006','RMB',3,300.00,300.00,'888888','否',6);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000007','RMB',3,200.00,200.00,'888888','否',7);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000008','RMB',3,150.00,150.00,'888888','否',8);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000009','RMB',3,100.00,100.00,'888888','否',9);
INSERT INTO cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000010','RMB',3,80.00,80.00,'888888','否',10);

```

插入完成后用查询语句查看各个表的情况，如图 11.10-12 所示。

```
SELECT * FROM deposit;
```

	savingid	savingname	descrip
1	1	活期	按存款日结算利息
2	2	定期一年	存款期是1年
3	3	定期二年	存款期是2年
4	4	定期三年	存款期是3年
5	5	定活两便	[NULL]
6	6	零存整取一年	存款期是1年
7	7	零存整取二年	存款期是2年
8	8	零存整取三年	存款期是3年
9	9	存本取息五年	按月支取利息

图 3.1 存款类型表

SELECT * FROM userinfo;

	customerid	customername	pid	telephone	address
1	1	John	110000000000000001	13900000001	江苏省南京市
2	2	Jane	110000000000000002	13900000002	[NULL]
3	3	Michael	110000000000000003	13900000003	[NULL]
4	4	Emily	110000000000000004	13900000004	[NULL]
5	5	David	110000000000000005	13900000005	[NULL]
6	6	Sarah	110000000000000006	13900000006	[NULL]
7	7	Robert	110000000000000007	13900000007	[NULL]
8	8	Jessica	110000000000000008	13900000008	[NULL]
9	9	William	110000000000000009	13900000009	[NULL]
10	10	Olivia	110000000000000010	13900000010	[NULL]

图 3.2 用户信息表

SELECT * FROM cardinfo;

	cardid	curid	savingid	opendate	openmoney	balance	pass	isreportloss	customerid
1	1010357600000001	RMB	1	2024-08-08 15:34:09	1000.00	1000.00	888888	否	1
2	1010357600000002	RMB	2	2024-08-08 15:40:08	1.00	1.00	888888	否	2
3	1010357600000003	RMB	2	2024-08-08 15:40:15	1.00	1.00	888888	否	3
4	1010357600000004	RMB	2	2024-08-08 15:40:21	1.00	1.00	888888	否	4
5	1010357600000005	RMB	3	2024-08-08 15:40:27	500.00	500.00	888888	否	5
6	1010357600000006	RMB	3	2024-08-08 15:40:33	300.00	300.00	888888	否	6
7	1010357600000007	RMB	3	2024-08-08 15:40:39	200.00	200.00	888888	否	7
8	1010357600000008	RMB	3	2024-08-08 15:40:45	150.00	150.00	888888	否	8
9	1010357600000009	RMB	3	2024-08-08 15:41:53	100.00	100.00	888888	否	9
10	1010357600000010	RMB	3	2024-08-08 15:41:58	80.00	80.00	888888	否	10

图 3.3 银行卡信息表

3.2 修改密码

修改客户密码要求如下。

修改 Olivia（卡号为 1010357600000010）银行卡密码为 123456。

修改 Emily（卡号为 1010357600000004）银行卡密码为 123123。

<pre> UPDATE cardinfo SET pass = '123456' WHERE cardid = '1010357600000010'; UPDATE cardinfo SET pass = '123123' WHERE cardid = '1010357600000004'; </pre>								
9	1010357600000010	RMB	3	2025-03-03 11:38:24	80.00	80.00	123456	否
10	1010357600000004	RMB	2	2025-03-03 11:38:24	1.00	1.00	123123	否

图 3.4 修改密码

3.3 挂失银行卡

Robert（卡号 1010357600000007）因银行卡丢失，申请挂失。

<pre> UPDATE cardinfo SET isreportloss = '是' WHERE cardid = '1010357600000007'; </pre>								
10	1010357600000007	RMB	3	2025-03-03 11:38:24	200.00	200.00	888888	是

图 3.5 银行卡挂失

3.4 问题分析

- （1）插入数据时，SQL 语法中注意拼写和中英文符号问题。
- （2）增加新数据时，注意表的约束关系。

4 实验进阶 3 银行业务数据封装

任务简介：

本次实验进阶主要是利用进阶 2 已经完成的数据库基础数据建设，实现对银行业务数据系统的封装，主要如下：

- （1）通过视图完成查询友好性（初级）；
- （2）通过存储过程实现模块化查询（中级）；
- （3）通过触发器实现自动开卡业务（高级）；
- （4）通过事务实现转账异常处理（高级）。

4.1 使用视图

为了向客户显示信息友好,查询各表要求字段全为中文字段名。

创建客户信息表视图，方便管理员直观查看每个客户的银行卡信息。

```
DROP VIEW IF EXISTS vw_userInfo;

CREATE VIEW vw_userInfo
AS
SELECT
    customerID AS "客户编号",
    customerName AS "开户名",
    PID AS "身份证号",
    telephone AS "电话号码",
    address AS "居住地址"
FROM userInfo;
```

查询客户信息表视图，如图 4.1 所示。

```
SELECT * FROM vw_userInfo;
```

	客户编号	开户名	身份证号	电话号码	居住地址
1	1	John	11000000000000000001	139000000001	江苏省南京市
2	2	Jane	11000000000000000002	139000000002	[NULL]
3	3	Michael	11000000000000000003	139000000003	[NULL]
4	4	Emily	11000000000000000004	139000000004	[NULL]
5	5	David	11000000000000000005	139000000005	[NULL]
6	6	Sarah	11000000000000000006	139000000006	[NULL]
7	7	Robert	11000000000000000007	139000000007	[NULL]
8	8	Jessica	11000000000000000008	139000000008	[NULL]
9	9	William	11000000000000000009	139000000009	[NULL]
10	10	Olivia	11000000000000000010	139000000010	[NULL]

图 4.1 用户信息视图

创建银行卡信息视图，方便管理员直观查看每个客户的银行卡信息。

```
CREATE VIEW vw_cardInfo
AS
SELECT
    c.cardID AS "卡号",
    u.customerName AS "客户姓名",
    c.curID AS "货币种类",
    d.savingName AS "存款类型",
    c.openDate AS "开户日期",
    c.balance AS "余额",
    c.pass AS "密码",
    CASE WHEN c.IsReportLoss = '否' THEN '正常' ELSE '挂失' END AS "是否挂失"
```



```
FROM cardInfo c
JOIN deposit d ON c.savingID = d.savingID
JOIN userInfo u ON c.customerID = u.customerID;
```

查询银行卡信息视图，如图 4.2 所示。

```
SELECT * FROM vw_cardInfo;
```

	卡号	客户姓名	货币种类	存款类型	开户日期	余额	密码	是否挂失
1	1010357600000001	John	RMB	活期	2024-08-26 15:32:32	1000.00	888888	正常
2	1010357600000002	Jane	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
3	1010357600000003	Michael	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
4	1010357600000005	David	RMB	定期二年	2024-08-26 15:32:32	500.00	888888	正常
5	1010357600000006	Sarah	RMB	定期二年	2024-08-26 15:32:32	300.00	888888	正常
6	1010357600000008	Jessica	RMB	定期二年	2024-08-26 15:32:32	150.00	888888	正常
7	1010357600000009	William	RMB	定期二年	2024-08-26 15:32:32	100.00	888888	正常
8	1010357600000010	Olivia	RMB	定期二年	2024-08-26 15:32:32	80.00	123456	正常
9	1010357600000004	Emily	RMB	定期一年	2024-08-26 15:32:32	1.00	123123	正常
10	1010357600000007	Robert	RMB	定期二年	2024-08-26 15:32:32	200.00	888888	挂失

图 4.2 银行卡信息视图

4.2 使用存储过程

任务 1：根据卡号查询余额

根据银行卡号查询该银行卡的余额，如果卡号不存在，则返回错误信息。

```
CREATE OR REPLACE PROCEDURE search_balance (
    p_cardid CHAR(19),
)
BEGIN
    -- 检查卡号是否存在
    IF NOT EXISTS (SELECT 1 FROM cardInfo WHERE cardID = p_cardid) THEN
        RAISE EXCEPTION '卡号不存在';
    END IF;

    -- 查询账户余额
    SELECT balance FROM cardInfo WHERE cardID = p_cardid;

    END IF;
END;
```

调用存储过程。

```
CALL search_balance ('1010357600000001'); --正常执行
CALL search_balance ('1010357600000002'); --正常执行
```

```
CALL search_balance ('1010357600000011'); --错误的卡号
```

查询结果为如下。

执行成功

执行成功

ERROR: 卡号不存在

2.打印客户对账单

客户在银行办理业务的时候经常需要打印对账单,即为某个特定的银行卡号打印指定时间内发生交易的对账单。

```
CREATE OR REPLACE PROCEDURE usp_CheckSheet(
    IN p_cardID VARCHAR(19),
    IN p_date1 TIMESTAMP DEFAULT NULL,
    IN p_date2 TIMESTAMP DEFAULT NULL
)
AS
DECLARE
    v_cardid VARCHAR(19);
    v_custName VARCHAR(20);
    v_curName VARCHAR(20);
    v_savingName VARCHAR(20);
    v_openDate TIMESTAMP;
    rec RECORD;
BEGIN
    SELECT c.cardID, c.curID, u.customerName, d.savingName, c.openDate
    INTO v_cardID, v_curName, v_custName, v_savingName, v_openDate
    FROM cardInfo c
    JOIN userInfo u ON c.customerID = u.customerID
    JOIN deposit d ON c.savingID = d.savingID
    WHERE c.cardID = p_cardID;

    RAISE NOTICE '卡号: ', v_cardID;
    RAISE NOTICE '姓名: ', v_custName;
    RAISE NOTICE '货币: ', v_curName;
    RAISE NOTICE '存款类型: ', v_savingName;
    RAISE NOTICE '开户日期: %年%月%日', TO_CHAR(v_openDate, 'YYYY'),
    TO_CHAR(v_openDate, 'MM'), TO_CHAR(v_openDate, 'DD');
    RAISE NOTICE ' ';
```

```

RAISE NOTICE '-----!';

FOR rec IN
    SELECT tradeDate AS "交易日", tradeType AS "类型", tradeMoney AS "交易金额", remark
AS "备注"

    FROM tradeInfo

    WHERE cardID = p_cardID AND tradeDate BETWEEN p_date1 AND p_date2

    ORDER BY tradeDate

LOOP
    RAISE NOTICE '%', rec;

END LOOP;

END IF;

END;

/

```

调用存储过程，如图 4.3 所示。

```

CALL usp_CheckSheet('1010357600000002');

CALL usp_CheckSheet('1010357600000001', '2024-08-25', '2024-08-27');

```

```

[NOTICE] 卡号: 1010357600000002
[NOTICE] 姓名: Jane
[NOTICE] 货币: RMB
[NOTICE] 存款类型: 定期一年
[NOTICE] 开户日期: 2024年08月08日
[NOTICE] -----
[NOTICE] ("2024-08-08 15:53:02.45374",存入,5000.00,)
[NOTICE] ("2024-08-08 16:05:04.042534",支取,1900.00,)
[NOTICE] ("2024-08-08 16:54:02.374043",支取,500.00,)

[NOTICE] 卡号: 1010357600000001
[NOTICE] 姓名: John
[NOTICE] 货币: RMB
[NOTICE] 存款类型: 活期
[NOTICE] 开户日期: 2024年08月08日
[NOTICE] -----
[NOTICE] ("2024-08-08 15:52:59.189046",支取,900.00,)
[NOTICE] ("2024-08-08 16:51:38.158797",存入,1000.00,)
[NOTICE] ("2024-08-08 16:56:57.774065",存入,1000.00,)

```

图 4.3 调用存储过程

4.3 使用触发器

创建新用户的时候，需要插入向用户信息表（userInfo）和银行卡信息表（cardInfo）插入用户信息和银行卡信息。为了方便操作，在创建新用户的同时，为其开户并且生成前四位为'10103576'后 8 位随机的卡号。

（1）创建触发器函数。

```
CREATE OR REPLACE FUNCTION create_card_for_new_user()
RETURNS TRIGGER AS $$
DECLARE
    card_number VARCHAR(16);
BEGIN
    LOOP
        -- 生成 8 位随机数字，并拼接前缀
        card_number := '10103576' || LPAD(FLOOR(RANDOM() * 100000000)::TEXT, 8, '0');

        -- 检查卡号是否已存在
        PERFORM 1 FROM cardInfo WHERE cardid = card_number;
        IF NOT FOUND THEN
            EXIT; -- 如果卡号唯一，退出循环
        END IF;
    END LOOP;

    -- 插入新卡号到 cardInfo 表
    INSERT INTO cardInfo (cardid, curid, savingid, openmoney, balance, pass, isreportloss, customerid)
    VALUES (
        card_number, -- 生成的随机卡号
        'RMB',       -- 默认币种
        1,           -- 默认储蓄类型
        1.00,        -- 默认开户金额（改为大于 0 的值）
        0.00,        -- 默认余额
        '888888',    -- 默认密码
        '否',        -- 默认是否挂失
        NEW.customerid -- 新用户的 customerid
    );
```

```

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

```

(2) 创建触发器。

```

CREATE TRIGGER trg_create_card

AFTER INSERT ON userInfo

FOR EACH ROW

EXECUTE PROCEDURE create_card_for_new_user();

```

(3) 创建完触发器后，创建一个新用户来进行测试。

```

INSERT INTO userInfo (customerid, customerName, PID, telephone, address)

VALUES (11, 'Alice', '1100000000000000011', '13900000011', '北京市海淀区');

```

(4) 查看用户信息表（userInfo）是否成功插入数据。

```

SELECT * FROM userinfo;

```

	customerid	customername	pid	telephone	address
1	1	John	1100000000000000001	13900000001	江苏省南京市
2	2	Jane	1100000000000000002	13900000002	[NULL]
3	3	Michael	1100000000000000003	13900000003	[NULL]
4	4	Emily	1100000000000000004	13900000004	[NULL]
5	5	David	1100000000000000005	13900000005	[NULL]
6	6	Sarah	1100000000000000006	13900000006	[NULL]
7	7	Robert	1100000000000000007	13900000007	[NULL]
8	8	Jessica	1100000000000000008	13900000008	[NULL]
9	9	William	1100000000000000009	13900000009	[NULL]
10	10	Olivia	1100000000000000010	13900000010	[NULL]
11	11	Alice	1100000000000000011	13900000011	北京市海淀区

图 4.4 查看用户信息表

(5) 查看银行卡信息表（cardInfo）是否成功为新用户开卡。

```

SELECT * FROM cardinfo;

```

	cardid	curid	savingid	opendate	openmoney	balance	pass	isreportloss	customerid
1	1010357600000000	RMB	1	2025-03-03 11:38:24	1000.00	1000.00	888888	否	1
2	10103576000000002	RMB	2	2025-03-03 11:38:24	1.00	1.00	888888	否	2
3	10103576000000003	RMB	2	2025-03-03 11:38:24	1.00	1.00	888888	否	3
4	10103576000000005	RMB	3	2025-03-03 11:38:24	500.00	500.00	888888	否	5
5	10103576000000006	RMB	3	2025-03-03 11:38:24	300.00	300.00	888888	否	6
6	10103576000000008	RMB	3	2025-03-03 11:38:24	150.00	150.00	888888	否	8
7	10103576000000009	RMB	3	2025-03-03 11:38:24	100.00	100.00	888888	否	9
8	10103576000000010	RMB	3	2025-03-03 11:38:24	80.00	80.00	123456	否	10
9	10103576000000004	RMB	2	2025-03-03 11:38:24	1.00	1.00	123123	否	4
10	10103576000000007	RMB	3	2025-03-03 11:38:24	200.00	200.00	888888	是	7
11	1010357675622805	RMB	1	2025-03-03 12:41:04	1.00	1.00	888888	否	11

图 4.5 查看银行卡信息表

作业：用户 John 的卡号（1010357600000001）取款 900 元，用户 Jane 的卡号（1010357600000002）存款 5000 元，要求保存交易记录，以便客户查询和银行业务统计。

说明：当存钱或取钱（如 900 元）时候，会往交易信息表（tradeInfo）中添加一条交易记录，同时应更新银行卡信息表（cardInfo）中的现有余额（如增加或减少 900 元）。

```
CREATE OR REPLACE FUNCTION update_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_current_balance NUMERIC(18,2);
BEGIN
    IF NEW.tradeType = '存入' THEN
        UPDATE cardInfo
        SET balance = balance + NEW.tradeMoney
        WHERE cardID = NEW.cardID;
    ELSIF NEW.tradeType = '支取' THEN
        SELECT balance INTO v_current_balance
        FROM cardInfo
        WHERE cardID = NEW.cardID;

        IF v_current_balance >= NEW.tradeMoney + 1 THEN
            UPDATE cardInfo
            SET balance = balance - NEW.tradeMoney
            WHERE cardID = NEW.cardID;
        ELSE
            RAISE EXCEPTION '余额不足';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_update_balance
BEFORE INSERT ON tradeInfo
FOR EACH ROW
EXECUTE PROCEDURE update_balance();
```

我们再尝试插入一些交易记录并查看银行卡信息表，如图 4.6 所示。

```
INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
```

VALUES('存入', '1010357600000005', 2000);

```
INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
```

VALUES('支取','1010357600000006',200);

9	1010357600000005	RMB	3	2024-08-08 15:40:27	500.00	2500.00	888888	否	5
10	1010357600000006	RMB	3	2024-08-08 15:40:33	300.00	100.00	888888	否	6

图 4.6 查看银行卡信息表

可以发现此时在不更新账户余额的情况下，账户余额会随着交易的正常执行而变化。

4.4 使用事务

用户 John（卡号为 1010357600000001）想要给用户 Jane（卡号为 1010357600000002）转账 500 元，可以使用一个事务完成此项任务。

创建事务如下。

BEGIN ;

INSERT INTO tradeInfo (tradeType, cardID, tradeMoney, remark)

VALUES ('支取', '1010357600000001', 500, '转账扣款');

INSERT INTO tradeInfo (tradeType, cardID, tradeMoney, remark)

VALUES ('存入', '1010357600000002', 500, '转账收款来自 John');

COMMIT;

查看交易信息表，如图 4.7 所示。

12	2024-08-26 20:37:09	支取	1010357600000001	500.00	转账扣款
13	2024-08-26 20:37:13	存入	1010357600000002	500.00	转账收款来自John

图 4.7 查看交易信息表结果

如果 John 转账 5000 元出现报错，则需要 ROLLBACK 回滚。

ROLLBACK;

4.5 问题分析

- (1) 生成银行卡号中卡号唯一性判别。
- (2) 事务中 Rollback 的使用。

5 实验进阶 4 银行业务数据安全性与 AI

任务简介：

本次实验进阶主要是利用进阶 3 已经完成的银行业务系统数据封装，实现对银行业务数据系统的安全管理与 AI 应用，主要如下：

- (1) 全密态数据库（高级）；
- (2) DB4AI（高级）。

5.1 全密态数据库

为了确保银行业务系统的数据安全和客户隐私安全，须采用全密态数据库和防篡改模式，保证数据在全生命周期中的加密状态，即使在数据传输、处理和存储环节遭遇攻击，也能确保攻击者无法获取任何有价值的信息。此外，防篡改功能通过记录和校验数据的操作历史，能有效防止数据被恶意篡改，确保了银行业务系统的完整性和可信度。

步骤 1：在 omm 用户下创建 \$GAUSSHOME/etc/localkms/ 路径。

```
mkdir -p $GAUSSHOME/etc/localkms/
```

步骤 2：omm 用户登录 bankdb 数据库，并开启密态开关。

```
gspl -d bankdb -p 26000 -r -C
```

步骤 3：在 bankdb 下创建防篡改模式 bank_security。

```
CREATE SCHEMA bank_security WITH BLOCKCHAIN;  
SET SEARCH_PATH TO bank_security;
```

步骤 4：创建客户端主密钥 CMK 和列加密密钥 CEK。

```
CREATE CLIENT MASTER KEY ImgCMK01 WITH (KEY_STORE = localkms, KEY_PATH =  
"key_path_value1", ALGORITHM = RSA_2048);  
CREATE CLIENT MASTER KEY ImgCMK02 WITH (KEY_STORE = localkms, KEY_PATH =  
"key_path_value2", ALGORITHM = RSA_2048);  
CREATE COLUMN ENCRYPTION KEY ImgCEK01 WITH VALUES (CLIENT_MASTER_KEY =  
ImgCMK01, ALGORITHM = AEAD_AES_256_CBC_HMAC_SHA256);  
CREATE COLUMN ENCRYPTION KEY ImgCEK02 WITH VALUES (CLIENT_MASTER_KEY =  
ImgCMK02, ALGORITHM = AEAD_AES_256_CBC_HMAC_SHA256);
```

步骤 5：在 bank_security 模式下创建加密表。

```
CREATE TABLE userInfo
```



```

(
    customerID SERIAL PRIMARY KEY,
    customerName CHAR(8) NOT NULL,
    PID CHAR(18) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK01,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL,
    telephone CHAR(20) NOT NULL,
    address VARCHAR(50)
);

CREATE TABLE cardInfo
(
    cardID CHAR(17) PRIMARY KEY,
    curID VARCHAR(10) DEFAULT 'RMB' NOT NULL,
    savingID INT NOT NULL DEFAULT 1,
    openDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    openMoney NUMERIC(18,2) NOT NULL,
    balance NUMERIC(18,2) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK01,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL,
    pass CHAR(6) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK02,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL DEFAULT '888888',
    IsReportLoss VARCHAR(3) NOT NULL DEFAULT '否',
    customerID INT NOT NULL
);

CREATE TABLE tradeInfo
(
    tradeDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tradeType CHAR(6) NOT NULL,
    cardID CHAR(19) NOT NULL,
    tradeMoney NUMERIC(18,2) NOT NULL,
    remark TEXT
);

CREATE TABLE deposit
(
    savingID SERIAL PRIMARY KEY,

```

```
savingName VARCHAR(20) NOT NULL,
descrip VARCHAR(50)
);
```

步骤 6：为表添加约束。因为密态数据库的特性，所以加密列不支持作为检查约束的一部分，对于加密列的约束可以在应用程序层面对明文进行验证，满足要求后再将其加密后存储到数据库中。

```
ALTER TABLE userInfo
    ADD CONSTRAINT chk_telephone CHECK (CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone
AS TEXT), '[^0-9]', '', 'g')) = 11);

ALTER TABLE cardInfo
    ADD CONSTRAINT ck_cardid CHECK (
        SUBSTRING(cardID, 1, 8) = '10103576'
        AND SUBSTRING(cardID, 9) ~ '^[0-9]{8}$'
    ),
    ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
    ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID) REFERENCES deposit(savingID);

ALTER TABLE tradeInfo
    ADD CONSTRAINT CK_tradeType CHECK(tradeType IN ('存入', '支取')),
    ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney > 0);
```

步骤 7：插入数据

插入数据部分的操作同用例 3。

查看 userinfo 表，可以看出在密态客户端查询时，数据仍以明文形式呈现，如图 5.1 所示。

```
SELECT * FROM userinfo;
```

```
bankdb=# SELECT * FROM userinfo;
 customerid | customername | pid | telephone | address
-----+-----+-----+-----+-----
          1 | John         | 11000000000000000001 | 13900000001 | 江苏省南京市
          2 | Jane         | 11000000000000000002 | 13900000002 |
          3 | Michael      | 11000000000000000003 | 13900000003 |
          4 | Emily        | 11000000000000000004 | 13900000004 |
          5 | David        | 11000000000000000005 | 13900000005 |
          6 | Sarah        | 11000000000000000006 | 13900000006 |
          7 | Robert       | 11000000000000000007 | 13900000007 |
          8 | Jessica      | 11000000000000000008 | 13900000008 |
          9 | William      | 11000000000000000009 | 13900000009 |
         10 | Olivia       | 11000000000000000010 | 13900000010 |
(10 rows)
```

图 5.1 在密态客户端查看用户信息表

bankdb=# SELECT * FROM gs_global_chain;									
blocknum	dbname	username	globalhash	txcommand	starttime	releid	relnsp	relname	relhash
1	bankdb	omm			2024-08-09 16:44:26.330344+08	16943	bank_security	deposit	fa409cdf29904606
9be755c8bda5d6e1de245ffd276748				INSERT INTO deposit (savingName,descrip) VALUES ('活期','按存款日结算利息');					d4
2	bankdb	omm			2024-08-09 16:44:26.331116+08	16943	bank_security	deposit	885167c015a6bf90
387501ad2e64d008bd6b4f6989af83				INSERT INTO deposit (savingName,descrip) VALUES ('定期一年','存款期是1年');					c9
3	bankdb	omm			2024-08-09 16:44:26.331642+08	16943	bank_security	deposit	5fef36418c89d7b0
242e71fad1fd4ba2e24b0639b41202				INSERT INTO deposit (savingName,descrip) VALUES ('定期二年','存款期是2年');					7e
4	bankdb	omm			2024-08-09 16:44:26.332146+08	16943	bank_security	deposit	beec190ef5817766
a9ff7ff1315e27244fc371fb3c51281				INSERT INTO deposit (savingName,descrip) VALUES ('定期三年','存款期是3年');					20
5	bankdb	omm			2024-08-09 16:44:26.332584+08	16943	bank_security	deposit	e585e3fc3c06656b
c89c2e877052b3f2b8bd4fb484eb54				INSERT INTO deposit (savingName) VALUES ('定活两便');					35
6	bankdb	omm			2024-08-09 16:44:26.333089+08	16943	bank_security	deposit	fa74f6517a550610
c49d29e5660c9b46a3487aada4428c				INSERT INTO deposit (savingName,descrip) VALUES ('零存整取一年','存款期是1年');					11
7	bankdb	omm			2024-08-09 16:44:26.333601+08	16943	bank_security	deposit	403ba37aaa936523
3fcec859c75bc7b92e8b12443e874f				INSERT INTO deposit (savingName,descrip) VALUES ('零存整取二年','存款期是2年');					dc
8	bankdb	omm			2024-08-09 16:44:26.334129+08	16943	bank_security	deposit	1a906f6dda3af2c0
11dc9ad7d6c8e76efbc3519d4a3569				INSERT INTO deposit (savingName,descrip) VALUES ('零存整取三年','存款期是3年');					28
9	bankdb	omm			2024-08-09 16:44:27.239666+08	16943	bank_security	deposit	b5f9e05bf74f7fb1
1a936c18993b3dddeb7a81c806cf239				INSERT INTO deposit (savingName,descrip) VALUES ('存本取息五年','按月支取利息');					42
10	bankdb	omm			2024-08-09 16:44:35.984039+08	16911	bank_security	cardinfo	a2eae534fc2ed20
4e418f653729034629feef6f8b8d5c				INSERT INTO cardInfo(cardID,savingID,openMoney,balance,customerID)					aa
				VALUES('1010357600000001',1,1000,'\x0185e0742662e039faa05868dc75097a653f9ebe505f82					
b906db3f8e5a215c2dabf128897a31000000db3b4ae17a8c481f70e4c66abc81be49f9a9ac21c2e5fad912da4b3b9077e0ae,1);									
11	bankdb	omm			2024-08-09 16:44:35.992583+08	16911	bank_security	cardinfo	7b9a002d5f649fe8
c40a7179dad9200be31db2859ec2df				INSERT INTO cardInfo(cardID,savingID,openMoney,balance,customerID)					dd

图 5.3 查看账本历史操作记录

作业：在全密态客户端中进行密码校验

创建一个存储过程，完成用户登录时对用户名和密码的校验，如果用户未注册或用户名和密码不匹配，则登录失败。

```

CREATE OR REPLACE PROCEDURE login(
    p_customer_name CHAR(8),
    p_pass CHAR(6),
)
AS
    v_id CHAR(8);
    v_pass CHAR(6);
BEGIN
    -- 检查用户名是否存在
    IF NOT EXISTS (SELECT 1 FROM userInfo WHERE userInfo.customerName = p_customer_name)
    THEN
        RAISE EXCEPTION '用户不存在，登录失败';
    END IF;

    -- 检查用户名和密码是否匹配
    SELECT customerID INTO v_id FROM userInfo WHERE userInfo.customerName = p_customer_name;

```

```

SELECT pass INTO v_pass FROM cardInfo WHERE cardInfo.customerID = v_id;

IF p_pass != v_pass THEN
    RAISE EXCEPTION '密码输入错误，登录失败';
ELSE
    RAISE NOTICE '登录成功';
END IF;

END;

/

```

5.2 DB4AI

DB4AI 是指利用数据库的能力驱动 AI 任务，实现数据存储、技术栈的同构。通过在数据库内集成 AI 算法，令 openGauss 具备数据库原生 AI 计算引擎、模型管理、AI 算子、AI 原生执行计划的能力，为用户提供普惠 AI 技术。不同于传统的 AI 建模流程，DB4AI “一站式”建模可以解决数据在各平台的反复流转问题，同时简化开发流程，并可通过数据库规划出最优执行路径，让开发者更专注于具体业务和模型的调优上，具备同类产品不具备的易用性与性能优势。

openGauss 支持原生 DB4AI 能力，通过引入原生 AI 算子，简化操作流程，充分利用数据库优化器、执行器的优化与执行能力，获得高性能的数据库内模型训练能力。更简化的模型训练与预测流程、更高的性能表现，让开发者在更短时间内能更专注于模型的调优与数据分析上，而避免了碎片化的技术栈与冗余的代码实现。

模型训练语法如下。

创建模型。

```
CREATE MODEL
```

以 multiclass 为例，训练一个模型。

从 tb_iris 训练集中指定 sepal_length, sepal_width, petal_length, petal_width 为特征列，使用 multiclass 算法，创建并保存模型 iris_classification_model。

```

CREATE MODEL iris_classification_model USING xgboost_regression_logistic FEATURES sepal_length,
sepal_width, petal_length, petal_width TARGET target_type < 2 FROM tb_iris_1 WITH nthread=4, max_depth=8;
MODEL CREATED. PROCESSED 1

```

查看模型信息：当训练完成后模型会被存储到系统表 gs_model_warehouse 中。系统表 gs_model_warehouse 可以查看到关于模型本身和训练过程的相关信息。

```
select * from gs_explain_model("model_name");
```

利用已存在的模型做推断任务：使用“SELECT”和“PREDICT BY”关键字利用已有模型完成推断任务。

```
SELECT...PREDICT BY...(FEATURES...)...FROM...;
```

查看执行计划：使用 explain 语句可对“CREATE MODEL”和“PREDICT BY”的模型训练或预测过程中的执行计划进行分析。Explain 关键字后可直接拼接 CREATE MODEL/PREDICT BY 语句（子句），也可接可选的参数。

示例 10.5 模拟房价预测模型

步骤 1：创建 house 表，包括房屋 ID、税费、卧室数量、浴室数量、价格、面积和地块面积等，如图 5.4 所示。

```
CREATE TABLE houses (id INT, tax INT, bedroom INT, bath FLOAT, price INT, size INT, lot INT);
INSERT INTO houses VALUES
(1, 590, 2, 1, 50000, 770, 22100),
(2, 1050, 3, 2, 85000, 1410, 12000),
(3, 20, 3, 1, 22500, 1060, 3500),
(4, 870, 2, 2, 90000, 1300, 17500),
(5, 1320, 3, 2, 133000, 1500, 30000),
(6, 1350, 2, 1, 90500, 820, 25700),
(7, 2790, 3, 2.5, 260000, 2130, 25000),
(8, 680, 2, 1, 142500, 1170, 22000),
(9, 1840, 3, 2, 160000, 1500, 19000),
(10, 3680, 4, 2, 240000, 2790, 20000),
(11, 1660, 3, 1, 87000, 1030, 17500),
(12, 1620, 3, 2, 118600, 1250, 20000),
(13, 3100, 3, 2, 140000, 1760, 38000),
(14, 2070, 2, 3, 148000, 1550, 14000),
(15, 650, 3, 1.5, 65000, 1450, 12000);
```

	id	tax	bedroom	bath	price	size	lot
1	1	590	2	1.0	50000	770	22100
2	2	1050	3	2.0	85000	1410	12000
3	3	20	3	1.0	22500	1060	3500
4	4	870	2	2.0	90000	1300	17500
5	5	1320	3	2.0	133000	1500	30000
6	6	1350	2	1.0	90500	820	25700
7	7	2790	3	2.5	260000	2130	25000
8	8	680	2	1.0	142500	1170	22000
9	9	1840	3	2.0	160000	1500	19000
10	10	3680	4	2.0	240000	2790	20000
11	11	1660	3	1.0	87000	1030	17500
12	12	1620	3	2.0	118600	1250	20000
13	13	3100	3	2.0	140000	1760	38000
14	14	2070	2	3.0	148000	1550	14000
15	15	650	3	1.5	65000	1450	12000

图 5.4 查看创建后的 house 表

步骤 2: 通过 CREATE MODEL 语句, 基于 SVM 算法创建一个二分类模型, 通过房屋税、浴室数量和房屋面积预测房屋价格是否低于 100000 元。

```
CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath, size TARGET price < 100000 FROM houses;
```

CREATE MODEL 语句用于模型的训练和保存。

USING 关键字指定算法名称。

FEATURES 用于指定训练模型的特征, 需根据训练数据表的列名添加。

TARGET 指定模型的训练目标, 它可以是训练所需数据表的列名, 也可以是一个表达式, 例如: price <100000。

步骤 3: 在 gs_model_warehouse 系统表中查看训练后的模型信息, 如图 5.5 所示, 仅截取最后的部分。

```
SELECT * FROM gs_model_warehouse WHERE modelname = 'house_binary_classifier';
```

trainingscoresname	trainingscoresvalue	modeldescr...
{accuracy,f1,precision,recall,loss}	{.666666687,.666666687,.625,.714285731,7.10693932}	[NULL]

图 5.5 在 gs_model_warehouse 系统表中查看训练后的模型信息

通过系统表的返回信息可以看到训练过程的预测准确率为 0.666667。

这对于二分类来说准确率并不高, 我们后面将尝试通过超参数来提升模型表现。

步骤 4: 使用 DROP MODEL 语句删除模型。

```
DROP MODEL house_binary_classifier;
```

步骤 5: 使用更优的超参数提高 SVM 算法的训练表现, 设置超参数:batch size 和学习率。

```
CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath, size TARGET price < 100000 FROM houses WITH batch_size=1, learning_rate=0.001;
```

再次查看训练后的模型信息如图 5.6 所示。

trainingscoresname	trainingscoresvalue
{accuracy,f1,precision,recall,loss}	{.733333349,.714285731,.714285731,.714285731,9.30504322}

图 5.6 查看训练后的模型信息

通过系统表的返回信息可以看到训练过程的预测准确率为 0.733333。

整体准确率要相对默认超参数有所提升。

步骤 6: 使用 PREDICT BY 语句预测样本数据。

创建一个新的字段, 表示价格是否小于 100000, 保存预测值的信息, 如图 5.7 所示。

```
SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY house_binary_classifier
```

```
(FEATURES tax, bath, size) AS price_pred FROM houses;
```

	tax	bath	size	price	price_actual	price_pred
1	590	1.0	770	50000	☒	☒
2	1050	2.0	1410	85000	☒	☒
3	20	1.0	1060	22500	☒	☒
4	870	2.0	1300	90000	☒	☒
5	1320	2.0	1500	133000	☐	☒
6	1350	1.0	820	90500	☒	☐
7	2790	2.5	2130	260000	☐	☐
8	680	1.0	1170	142500	☐	☒
9	1840	2.0	1500	160000	☐	☐
10	3680	2.0	2790	240000	☐	☐
11	1660	1.0	1030	87000	☒	☐
12	1620	2.0	1250	118600	☐	☐
13	3100	2.0	1760	140000	☐	☐
14	2070	3.0	1550	148000	☐	☐
15	650	1.5	1450	65000	☒	☒

图 5.7 创建一个新的字段保存预测值的信息

步骤 7: 通过 CREATE MODEL 语句创建一个逻辑回归 (logistic regression) 模型, 通过房屋税、浴室数量和房屋面积预测房屋价格是否低于 100000 元。

```
CREATE MODEL house_logistic_classifier USING logistic_regression FEATURES tax, bath, size target
price < 100000 FROM houses WITH learning_rate=0.001;
```

步骤 8: 利用训练好的逻辑回归模型预测数据, 并与 SVM 算法进行比较, 如图 5.8 所示。

```
SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY house_binary_classifier
(FEATURES tax, bath, size) AS price_svm_pred, PREDICT BY house_logistic_classifier (FEATURES tax, bath,
size) AS price_logistic_pred FROM houses;
```

	tax	bath	size	price	price_actual	price_svm...	price_logist...
1	590	1.0	770	50000	☒	☒	☒
2	1050	2.0	1410	85000	☒	☒	☒
3	20	1.0	1060	22500	☒	☒	☒
4	870	2.0	1300	90000	☒	☒	☒
5	1320	2.0	1500	133000	☐	☒	☒
6	1350	1.0	820	90500	☒	☐	☐
7	2790	2.5	2130	260000	☐	☐	☐
8	680	1.0	1170	142500	☐	☒	☒
9	1840	2.0	1500	160000	☐	☐	☐
10	3680	2.0	2790	240000	☐	☐	☐
11	1660	1.0	1030	87000	☒	☐	☐
12	1620	2.0	1250	118600	☐	☐	☐
13	3100	2.0	1760	140000	☐	☐	☐
14	2070	3.0	1550	148000	☐	☐	☐
15	650	1.5	1450	65000	☒	☒	☒

图 5.8 训练好的逻辑回归模型预测数据与 SVM 算法模型比较结果

5.3 问题分析

- (1) 列加密密钥必须和主密钥一一匹配。
- (2) 在 DB4AI 中，删除已创建的模型时，必须给当前用户赋予 Drop 权限。

6 实验进阶 5 性能调优与社区贡献

任务简介：

本次实验进阶主要是利用进阶 4 已经完成的相关业务，实现对银行业务数据系统的性能调优与社区贡献，主要如下：

- (1) 系统性能调优（高级）；
- (2) 社区贡献（高级）。

6.1 系统性能调优

任务 6.1：通过分区表实现查询性能调优

当银行业务系统中的数据存储量达到一定规模时，可以使用分区表存储数据，提升查询时的效率。普通表若要转成分区表，需要新建分区表，然后把普通表中的数据导入到新建的分区表中。因此在初始设计表时，需要根据业务提前规划是否使用分区表。

在创建交易信息表时，可以根据交易时间字段进行分区。

```
CREATE TABLE tradeInfo
(
    tradeDate  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tradeType  CHAR(6) NOT NULL,
    cardID    CHAR(19) NOT NULL,
    tradeMoney NUMERIC(18,2) NOT NULL,
    remark    TEXT
);
PARTITION BY RANGE(tradeDate)
(
    PARTITION season1 VALUES LESS THAN('2024-04-01 00:00:00'),
    PARTITION season2 VALUES LESS THAN('2024-07-01 00:00:00'),
    PARTITION season3 VALUES LESS THAN('2024-10-01 00:00:00'),
    PARTITION season4 VALUES LESS THAN(MAXVALUE)
);
```

创建分区表后，插入用例 3 中的数据，在数据插入成功后，我们进行分区表的查询，查询分区表的语法格式如下：

```
SELECT * FROM partition_table_name PARTITION { ( partition_name ) | FOR ( partition_value  
[ , ... ] ) }
```

其中，partition_table_name 是分区表的名称，partition_name 为分区的名称，partition_value 用于指定分区的值。

想要查询在 2024 年 10 月 1 日之后进行的交易记录，使用的查询 sql 语句如下。

```
SELECT * FROM tradeInfo PARTITION (season4);
```

在查询分区表中特定的分区时，range 分区能够进行分区裁剪，最终只扫描一个分区就能将结果返回，可以有效提升查询效率。

任务 6.2：通过简单的索引实现查询性能调优

索引是提高查询性能的重要方法。索引利用表的某些常用的查询列的数据构建树形结构，以空间换时间的手段，优化每次查询的性能。

例如，为银行卡信息表的用户 ID 字段添加索引，按照用户 ID 字段对银行卡信息表进行查询时的效率将提高。

```
openGauss=# create index idx_cardinfo_id on cardInfo(customerID);  
  
CREATE INDEX
```

当存储的数据量达到一定规模时，使用索引后，查询语句的执行速度会提高一个数量级以上。数据量越大，执行速度的提升越明显。

任务 6.3：通过参数调整实现查询性能调优

openGauss 的 SMP 并行技术是一种利用计算机多核 CPU 架构来实现多线程并行计算，以充分利用 CPU 资源来提高查询性能的技术。

使用 SMP 并行技术优化 openGauss 查询效率的典型操作是调整 openGauss 系统中的 query_dop 参数，调整流程如下。

(1) 重置 query_dop 参数并查询当前 query_dop 参数的数值。

```
openGauss=# reset query_dop;  
  
RESET  
  
openGauss=# show query_dop;  
  
query_dop  
-----  
  
1
```

(2) 将 query_dop 设置为 4，开启并行执行

```
openGauss=# set query_dop = 4;
```

SET

比较 `query_dop` 取值不同时的查询耗时，可以发现当数据量达到一定规模时，开启并行执行后，耗时比普通模式少得多。

任务 6.4：通过子查询优化实现查询性能调优

在 openGauss 中，子查询分为相关子查询和非相关子查询，相关子查询无法进行调优，耗时较长，因此在设计子查询时，应尽量避免相关子查询，在可能的情况下尽量将其改造为非相关子查询。以下通过案例演示相关子查询和非相关子查询的差异。

（1）开启耗时统计开关

```
openGauss=# \timing on
Timing is on.
```

耗时统计开关开启后，即可对比不同查询语句的耗时差异。

（2）非相关子查询

```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.b in (1, 2, 3, 4, 5));
```

（3）相关子查询

```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.a = t1.a and t2.b in (1, 2, 3, 4, 5));
```

6.2 社区贡献

openGauss 用户可以通过多种方式协助 openGauss 社区。其中最为常用的是通过提交代码来参与社区贡献，下面将通过以下步骤指引带领认证者掌握如何向 openGauss 社区提交 PR。

社区实践题目作答并提交 PR

进入 Clone 的代码目录，拷贝 examination 目录并命名为 examinationyourname 形式，如图 6.1 所示。

```
opengauss@localhost ~]$ cd openGauss-sandbox/
opengauss@localhost openGauss-sandbox]$
opengauss@localhost openGauss-sandbox]$ cp -rf examination examination_nelson
```

图 6.1 Clone 代码目录

进入新拷贝的目录，打开 question 文件并进行作答

作答完成后，使用类似如下命令提交作答的结果到自己的私仓，注意修改用户名

git add examination_nelson

git commit -m "commit nelson sandbox test results"

git push origin master

提交作答结果如图 6.2 所示。

```
[opengauss@localhost openGauss-sandbox]$ git add examination_nelson
[opengauss@localhost openGauss-sandbox]$ git commit -m "commit nelson sandbox test results"
[master a2f51c5] commit nelson sandbox test results
1 file changed, 21 insertions(+)
create mode 100644 examination_nelson/question
[opengauss@localhost openGauss-sandbox]$ git push origin master
Username for 'https://gitee.com': Nelson-He
Password for 'https://Nelson-He@gitee.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 279 bytes | 279.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Powered by GITEE.COM [GNK-6.4]
To https://gitee.com/Nelson-He/openGauss-sandbox.git
fb8381b..a2f51c5 master -> master
[opengauss@localhost openGauss-sandbox]$
```

图 6.2 提交作答结果

接下来在浏览器中打开自己的私仓地址，单击“+Pull Request”按钮创建提交 PR，如图 6.3 所示。



图 6.3 创建提交 PR

在弹出的 PR 创建页面填写 PR 的标题和说明部分，如图 6.4 所示。

创建 Pull Request

源分支: Nelson-He/openGauss-sand... master

目标分支: opengauss/openGauss-sandbox master

✓ 可自动合并

Nelson同学提交openGauss沙箱实验认证题答案

B I H [icons] [editor icon] [help icon]

Nelson同学提交openGauss沙箱实验认证题答案

图 6.4 PR 页面填写

填完 PR 标题和内容后，在右侧界面将第 3 步提交的 Issue 进行关联，这样 PR 被接受后 Issue 会被关联关闭，如图 6.5 所示。



图 6.5 关联 less

上面的步骤完成后，单击页面下方的“创建 Pul Request”按钮即可，如图 6.6 所示。

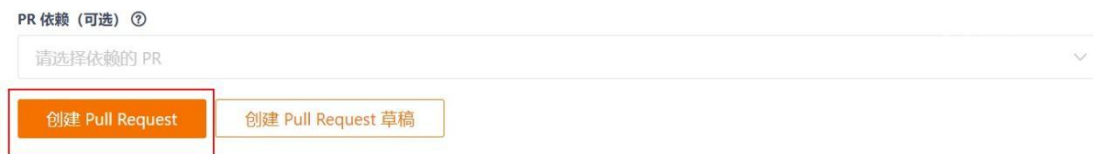


图 6.6 创建 PR

6.3 问题分析

- (1) 设计非相关子查询时注意其语法结构。
- (2) 在向 openGauss 社区推送代码时，要注意 git 命令语法。

附录

附录 1 openGauss 数据库安装

(1) 安装前准备

在 openGauss 安装前，需要准备相应的软硬件环境以及完成相关配置。

目前，openGauss 支持的 Linux 操作系统如下。

- 1) openEuler-20.03-LTS (aarch64 架构)
- 2) openEuler-20.03-LTS (x86 架构)
- 3) CentOS 7.6 (x86 架构)

表 1 列出了 openGauss 服务器应具备的最低硬件要求。在实际产品中，硬件配置的规划需考虑数据规模及所期望的数据库响应速度。请根据实际情况进行规划。

表 1 openGauss 服务器应具备的最低硬件要求

项目	配置描述
----	------

内存	功能调试建议 32GB 以上。 性能测试和商业部署时，单实例部署建议 128GB 以上。 复杂的查询对内存的需求量比较高，在高并发场景下，可能出现内存不足。此时建议使用大内存的机器，或使用负载管理限制系统的并发。
CPU	功能调试最小 1×8 核 2.0GHz。 性能测试和商业部署时，单实例部署建议 1×16 核 2.0GHz。 CPU 超线程和非超线程两种模式都支持。 说明：个人开发者最低配置 2 核 4G, 推荐配置 4 核 8G。目前，openGauss 仅支持鲲鹏服务器和基于 X86_64 通用 PC 服务器的 CPU。
硬盘	用于安装 openGauss 的硬盘需最少满足如下要求： 至少 1GB 用于安装 openGauss 的应用程序。 每个主机需大约 300MB 用于元数据存储。 预留 70%以上的磁盘剩余空间用于数据存储。 建议系统盘配置为 Raid1，数据盘配置为 Raid5，且规划 4 组 Raid5 数据盘用于安装 openGauss。有关 Raid 的配置方法在本手册中不做介绍。请参考硬件厂家的手册或互联网上的方法进行配置，其中 Disk Cache Policy 一项需要设置为 Disabled，否则机器异常掉电后有数据丢失的风险。 openGauss 支持使用 SSD 盘作为数据库的主存储设备，支持 SAS 接口和 NVME 协议的 SSD 盘，以 RAID 的方式部署使用。
网络要求	300 兆以上以太网。 建议网卡设置为双网卡冗余 bond。

目前 openGauss 数据库仅支持在防火墙关闭的状态下进行安装，所以需要先关闭操作系统防火墙，另外用户还需要将同 openGauss 相关的一些服务、协议、IP 以及端口添加到 openGauss 各主机的防火墙的白名单当中。

（2）获取并校验安装包

openGauss 数据库的安装包需要在 openGauss 开源社区下载。首先登录 openGauss 开源社区，然后选择对应平台的符合个人需求的 openGauss 安装包并下载。下载完成后，要对安装包内容进行检查和解压。

（3）配置 XML 文件

在安装 openGauss 之前，需要创建 cluster_config.xml 文件。cluster_config.xml 文件包含部署 openGauss 的服务器信息、安装路径、IP 地址以及端口号等。用于告知 openGauss 如何部署。用户需要根据不同场景配置对应的 XML 文件。

在 script/gspylib/etc/conf/cluster_config_template.xml 路径下获取 XML 文件模板。以下配置内容为示例，实际的配置以个人情况为准进行自定义。每行信息均有注释进行说明。

1）配置数据库名称及各项目录

```
<?xml version="1.0" encoding="UTF-8"?>

<ROOT>
```

```

<!-- 整体信息 -->

<CLUSTER>

<!-- 数据库名称 -->

    <PARAM name="clusterName" value="dbCluster" />

<!-- 数据库节点名称(hostname) ， 填写个人实际主机名称-->

    <PARAM name="nodeNames" value="ecs-xxxx" />

<!-- 数据库安装目录-->

    <PARAM name="gaussdbAppPath" value="/opt/gaussdb/app" />

<!-- 日志目录-->

    <PARAM name="gaussdbLogPath" value="/var/log/gaussdb" />

<!--数据库工具目录-->

    <PARAM name="gaussdbToolPath" value="/opt/huawei/wisquery" />

<!--数据库 core 文件目录-->

    <PARAM name="corePath" value="/opt/opengauss/corefile"/>

<!-- 节点 IP， 与 nodeNames 一一对应， 根据个人实际 IP 地址填写 -->

    <PARAM name="backIp1s" value="192.168.0.xxx"/>

</CLUSTER>

```

2) 配置 Host 基本信息

```

<!-- 每台服务器上的节点部署信息 -->

<DEVICELIST>

<!-- 节点 1 上的部署信息 -->

<DEVICE sn="1000001">

<!-- 节点 1 的主机名称 -->

    <PARAM name="name" value="ecs-xxxx" />

<!-- 节点 1 所在的 AZ 及 AZ 优先级 -->

    <PARAM name="azName" value="AZ1"/>

    <PARAM name="azPriority" value="1"/>

<!-- 节点 1 的 IP， 如果服务器只有一个网卡可用， 将 backIP1 和 sshIP1 配置成同一个 IP -->

    <PARAM name="backIp1" value="192.168.0.xxx"/>

    <PARAM name="sshIp1" value="192.168.0.xxx"/>

```

3) 配置数据库主节点信息

```

<!--DBnode-->

<PARAM name="dataNum" value="1"/>

<!--数据库端口号-->

<PARAM name="dataPortBase" value="5432"/>

```

```
<!--数据库主节点上的数据目录-->
<PARAM name="dataNode1" value="/gaussdb/data/db1" />
</DEVICE>
</DEVICELIST>
</ROOT>
```

(4) 初始化安装环境

为了保证 openGauss 的正确安装，首先要对主机环境进行配置。

创建完 openGauss 配置文件后，在执行安装前，为了后续能以最小权限进行安装及 openGauss 管理操作，保证系统安全性，需要运行安装前置脚本 gs_preinstall 准备好安装用户及环境。gs_preinstall 命令只能使用 root 用户执行。

准备安装用户及环境的步骤如下。

1) 以 root 用户登录待安装 openGauss 的任意主机，并按规划创建存放安装包的目录。

```
mkdir -p /opt/software/openGauss
chmod 755 -R /opt/software
```

2) 将安装包“openGauss-x.x.x-openEuler-64bit-all.tar.gz”和配置文件“cluster_config.xml”都上传至上一步所创建的目录中。

3) 在安装包所在的目录下，解压安装包，解压完成后，会在/opt/software/openGauss 路径下自动生成 script 子目录，并且在 script 目录下生成 gs_preinstall 等各种 OM 工具脚本。

```
cd /opt/software/openGauss
tar -zxvf openGauss-x.x.x-openEuler-64bit-all.tar.gz
tar -zxvf openGauss-x.x.x-openEuler-64bit-om.tar.gz
```

4) 进入到工具脚本存放目录下。

```
cd /opt/software/openGauss/script
```

5) 如果是 openEuler 的操作系统，为确保适配 python 版本，执行如下命令，打开 gspylib/common/CheckPythonVersion.py 文件。

```
vi gspylib/common/CheckPythonVersion.py
```

将 if not pythonVersion == (3, 6):修改为 if not pythonVersion >= (3, 6):，然后按下“ESC”键进入指令模式，执行“:wq”保存修改并退出。

6) 如果是 openEuler 的操作系统，执行如下命令，打开 performance.sh 文件。

```
vi /etc/profile.d/performance.sh
```

用#符号注释 sysctl -w vm.min_free_kbytes=112640 &> /dev/null，然后按下“ESC”键进入指令模式，执行“:wq”保存修改并退出。

7) 为确保成功安装，检查 hostname 与/etc/hostname 是否一致。预安装过程中，会对 hostname 进行检查。

8) 使用 gs_preinstall 准备好安装环境。

```
./gs_preinstall -U omm -G dbgrp -X /opt/software/openGauss/cluster_config.xml
```

(5) 执行安装

使用 gs_install 命令安装 openGauss 数据库的操作步骤如下。

1) 检查安装包和 openGauss 配置文件在规划路径下是否已存在，如果没有，重新执行预安装，确保预安装成功。

2) 登录到 openGauss 的主机，并切换到 omm 用户。

```
su - omm
```

3) 使用 gs_install 安装 openGauss。

```
gs_install -X /opt/software/openGauss/cluster_config.xml
```

其中，/opt/software/openGauss/cluster_config.xml 为 openGauss 配置文件的路径。在执行过程中，用户需根据提示输入数据库的密码，密码具有一定的复杂度，为保证用户正常使用该数据库，请记住输入的数据库密码。设置的密码要符合 openGauss 的复杂度要求。

(6) 安装完成后生成的目录

安装完成后，openGauss 的相关目录说明参见表 2-4。

表 2-4 openGauss 安装后的目录说明

序号	项目目录说明	目录	说明
1	集群 openGauss 安装目录	/opt/gaussdb/app	存放数据库运行所需要的文件
2	集群 openGauss 数据目录	/gaussdb/data	存放数据库实例
3	集群 openGauss 日志目录	/var/log/gaussdb/用户名	存放数据库运行中产生的日志文件
4	集群 openGauss 系统工具目录	/opt/Huawei/wisquery	存放 openGauss 管理的脚本文件和二进制文件依赖的库文件

附录 2 DataStudio 安装

步骤 1 Data Studio 图形界面客户端依赖 JDK，请先在客户端主机上安装 JDK。在 Windows 操作系统中，您可以访问 JDK 官网网站，下载符合操作系统版本的 JDK，并根据指导进行安装。

步骤 2 登录 GaussDB(DWS)管理控制台。在左侧导航栏中，单击“管理 > 连接客户端”，进入“下载客户端和驱动”页面。

在“下载客户端和驱动”页面，下载“Data Studio 图形界面客户端”。

请根据操作系统类型，选择“Windows x86”或“Windows x64”，再单击“下载”，可以下载与现有集群版本匹配的 Data Studio 工具。

单击“历史版本”可根据集群版本下载相应版本的 Data Studio 工具，建议按集群版本下载配套的工具。

下载客户端和驱动

下载客户端 ②

命令行客户端

Microsoft Windows

下载

校验文件

历史版本

Windows Server 2008/Windows 7及以上

Data Studio 图形界面客户端

Microsoft Windows x64

下载

校验文件

历史版本

Data Studio安装前需要正确安装java8, 并请确认Data Studio、java8和电脑操作系统的位数一致。

DSC语法转换工具

DSC_8.3.1.111.zip

下载

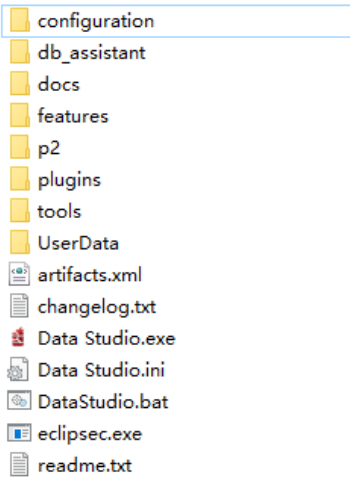
校验文件

DSC安装前需要正确安装jdk8, 并请确认java8和电脑操作系统的位数一致。

如果同时拥有不同版本的集群，选择与集群版本相对应的 gsql 版本客户端。在“集群管理”页面的集群列表中，单击指定集群的名称，再选择“集群详情”页签，可查看集群版本。

步骤 3 解压下载的客户端软件包（32 位或 64 位）到需要安装的路径。

步骤 4 打开安装目录，双击 Data Studio.exe，启动 Data Studio 客户端。

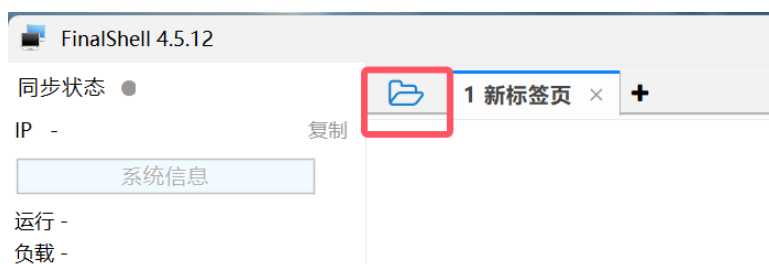


附录 3 FinalShell 安装与建立连接

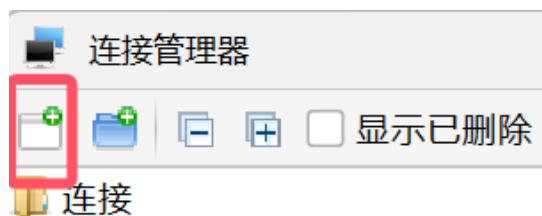
步骤 1 进入 FinalShell 的下载网址 <https://www.hostbuf.com/t/988.html>，点击 Windows x64 版的下载地址下载安装包。下载完毕后，进行一键式安装。



步骤 2 启动 FinalShell，单击蓝色文件夹图标，打开连接管理器。



步骤 3 单击白色文件夹图标并选择“SSH 连接”，建立新的连接。



步骤 4 在新建连接页面，名称可自定义，主机填写华为云 ECS 服务器的公网 IP 地址，端口默认为 22，用户名为 root，密码为服务器 root 用户的密码，填写完毕后点击确定即可与远程服务器建立连接。

新建连接

SSH连接

终端

代理服务器

隧道

常规

名称:

主机:端口:

22

备注:

认证

方法:

密码

用户名:

密码:

私钥:

浏览...

高级

☐ 智能加速 (加速海外服务器连接)

☒ 启用Exec Channel(若连接上就被断开,请关闭该项,比如跳板机)

关闭后无法监控服务器信息

确定

取消